Probleme_microbiologie_SUBRA_ileana

2026-10-29

```
#Lecture des données
srs_pop <-read.table(file='Detection_Table(1).csv', header=FALSE)
nbgr <- dim(srs_pop)[1]
nbsp <- 150 # le nombre d'espèces, ici 150.
```

installation du package sampling pour effectuer différents types d'échantillonnages.

```
library(sampling)
```

```
## Warning: le package 'sampling' a été compilé avec la version R 4.4.3
```

```
set.seed(2026)
#crée les échantillons
ech_sans_rem <- srswor(nbsp,nbgr)
# On fait un tirage aléatoire simple sans remise, chaque espèce ne peut être
# sélectionné qu'une fois.
```

```
sample_e <- as.vector(srs_pop$V1[ech_sans_rem==1])
# sous forme de vecteur on
#extrait les individus pour lesquels l'indicateur = 1. Si =1 l'individu est
#contaminé si =0, il ne l'est pas.
```

```
# calcul de la contamination
prop_pop <- sum(srs_pop$V1)/nbgr
# proportion d'individus contaminés dans la population avec srs_pop$V1 le
# nombre d'individus contaminés.
p_ech <- sum(sample_e)/nbsp
# proportion d'individus contaminés dans l'échantillon.
# on calcul les deux afin de les comparer car dans la réalité on connait
# rarement la proportion de contamination dans une population.
```

```
# on établi un intervalle de confiance bilatéral à 95%
t_ech <- qt(0.025,149,lower.tail = FALSE, log.p = FALSE)
# qt calcul le quantile de la loi de student, le degré de liberté correspond à
# la taille de l'échantillon-1
v_ech <- p_ech*(1.0-p_ech)*(1.0-nbsp/nbgr)
# on calcul la variance pour déterminer l'incertitude dû à la proportion de
# l'échantillon. On multiplie par le facteur de correction de la variance
# sans remise
mu_min <- p_ech-t_ech*sqrt(v_ech/nbsp)
mu_max <- p_ech+t_ech*sqrt(v_ech/nbsp)
#mu_min et mu_max sont les bornes minimales et maximales de l'intervalle.
```

```
print(c(mu_min,p_ech,mu_max))
```

```
## [1] 0.1125502 0.1733333 0.2341165
```

```
# Pour visualiser l'intervalle et la proportion de contamination calculé.
```

On essaye d'estimer le nombre d'individus contaminé à partir de la proportion de contamination de l'échantillon. Pour cela, on multiplie la proportion par la taille de la population, le résultat est arrondi à l'entier le plus proche.

```
N_ech <- round((sum(sample_e)/nbsp)*nbgr)
N_pop <- sum(srs_pop$V1)
# calcul le nombre d'individus contaminés dans la population.
V_ech_2 <- p_ech*(1.0-p_ech)*(1.0-nbsp/nbgr)*nbgr^2
# on calcul la variance de l'estimation ainsi que son intervalle.
# On définit Mu_min et Mu_max comme les bornes supérieures et inférieures
# de l'intervalle.
Mu_min <- N_ech-t_ech*sqrt(V_ech_2/nbsp)
Mu_max <- N_ech+t_ech*sqrt(V_ech_2/nbsp)
print(c(Mu_min,N_ech,Mu_max))
```

```
## [1] 1781.681 2744.000 3706.319
```

```
# On fait un tirage mais cette fois avec remise.
ech_rem <- srswr(nbsp,nbgr)
sample_r <- ech_rem [ech_rem>=1] # comme un individus peut être sélectionné
# plusieurs fois, on a désormais un signe >= et non plus =1. calcul le  nombre
# de fois où les individus sélectionnés l'ont été.
print(sample_r)
```

```
##   [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
##  [38] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
##  [75] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [112] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [149] 1 1
```

```
sample_u <- srs_pop$V1[ech_rem>=1] # on calcul le nombre d'individus contaminés
# parmis ceux sélectionnés. Si un 1 s'affiche, l'individu est contaminé si
# c'est=0 alors il ne l'est pas.
sample_v <- 1:nbsp # on crée un vecteur qui va contenir les observations des
# 150 tirages effectués dans la boucle qui va suivre.
index <- 1 # précise la position de la prochaine observation.
for (i in 1:length(sample_r)) { # le nombre d'individus qui ont été sélectionné
  sgr <- rep(sample_u[i],sample_r[i])
  # on répète le nombre de fois que chaque individus à été sélectionné, avec i
  # le nombre de répétition.
  sample_v[index:(index+sample_r[i]-1)]<- sgr
  # Cette ligne nous permet d'ordonner les tirages de 1 à 150
  index <- index+sample_r[i] # on assigne la position suivante
  print(c(i,index,sgr)) # Nous montre les résultats ordonnés.
}
```

```
## [1] 1 2 0
## [1] 2 3 0
## [1] 3 4 0
## [1] 4 5 0
## [1] 5 6 1
## [1] 6 7 1
## [1] 7 8 0
## [1] 8 9 0
```

```
## [1]  9 10  0
## [1] 10 11  0
## [1] 11 12  0
## [1] 12 13  0
## [1] 13 14  0
## [1] 14 15  0
## [1] 15 16  0
## [1] 16 17  0
## [1] 17 18  0
## [1] 18 19  0
## [1] 19 20  0
## [1] 20 21  1
## [1] 21 22  0
## [1] 22 23  0
## [1] 23 24  0
## [1] 24 25  0
## [1] 25 26  1
## [1] 26 27  0
## [1] 27 28  0
## [1] 28 29  0
## [1] 29 30  0
## [1] 30 31  1
## [1] 31 32  0
## [1] 32 33  0
## [1] 33 34  0
## [1] 34 35  0
## [1] 35 36  0
## [1] 36 37  1
## [1] 37 38  1
## [1] 38 39  0
## [1] 39 40  0
## [1] 40 41  0
## [1] 41 42  0
## [1] 42 43  0
## [1] 43 44  1
## [1] 44 45  0
## [1] 45 46  1
## [1] 46 47  0
## [1] 47 48  0
## [1] 48 49  1
## [1] 49 50  0
## [1] 50 51  0
## [1] 51 52  1
## [1] 52 53  0
## [1] 53 54  1
## [1] 54 55  0
## [1] 55 56  1
## [1] 56 57  1
## [1] 57 58  0
## [1] 58 59  0
## [1] 59 60  1
## [1] 60 61  1
## [1] 61 62  0
## [1] 62 63  0
```

```
## [1] 63 64  0
## [1] 64 65  1
## [1] 65 66  0
## [1] 66 67  0
## [1] 67 68  0
## [1] 68 69  0
## [1] 69 70  1
## [1] 70 71  1
## [1] 71 72  0
## [1] 72 73  0
## [1] 73 74  1
## [1] 74 75  0
## [1] 75 76  1
## [1] 76 77  0
## [1] 77 78  0
## [1] 78 79  0
## [1] 79 80  1
## [1] 80 81  0
## [1] 81 82  0
## [1] 82 83  1
## [1] 83 84  1
## [1] 84 85  0
## [1] 85 86  0
## [1] 86 87  0
## [1] 87 88  0
## [1] 88 89  0
## [1] 89 90  0
## [1] 90 91  1
## [1] 91 92  0
## [1] 92 93  0
## [1] 93 94  0
## [1] 94 95  1
## [1] 95 96  0
## [1] 96 97  0
## [1] 97 98  1
## [1] 98 99  0
## [1]  99 100   1
## [1] 100 101   0
## [1] 101 102   0
## [1] 102 103   1
## [1] 103 104   0
## [1] 104 105   0
## [1] 105 106   0
## [1] 106 107   0
## [1] 107 108   0
## [1] 108 109   0
## [1] 109 110   0
## [1] 110 111   1
## [1] 111 112   0
## [1] 112 113   0
## [1] 113 114   1
## [1] 114 115   1
## [1] 115 116   1
## [1] 116 117   0
```

```
## [1] 117 118   0
## [1] 118 119   1
## [1] 119 120   0
## [1] 120 121   0
## [1] 121 122   1
## [1] 122 123   0
## [1] 123 124   0
## [1] 124 125   0
## [1] 125 126   0
## [1] 126 127   0
## [1] 127 128   1
## [1] 128 129   0
## [1] 129 130   0
## [1] 130 131   0
## [1] 131 132   0
## [1] 132 133   0
## [1] 133 134   1
## [1] 134 135   0
## [1] 135 136   0
## [1] 136 137   1
## [1] 137 138   0
## [1] 138 139   0
## [1] 139 140   0
## [1] 140 141   0
## [1] 141 142   0
## [1] 142 143   1
## [1] 143 144   0
## [1] 144 145   0
## [1] 145 146   1
## [1] 146 147   1
## [1] 147 148   0
## [1] 148 149   0
## [1] 149 150   0
## [1] 150 151   0
```

```r
sample_e <- as.vector(sample_v)
p_ech_r_2 <- sum(sample_e)/nbsp
# On calcul la proportion de contamination dans notre échantillon. Ici 0.28 donc
# une proportion de contamination estimée à 28%
v_ech_r <- p_ech_r_2*(1.0-p_ech_r_2) # On calcul la variance de la proportion de
# contamination. Ici 0.20, variance estimée à 20%
```

```r
# Lire les données
srs_pop <- read.table("Detection_Table(1).csv", header = FALSE)

# On extrait chaque groupe
grp_A <- srs_pop[srs_pop$V2 == "A", ]
grp_B <- srs_pop[srs_pop$V2 == "B", ]
grp_N <- srs_pop[srs_pop$V2 == "N", ]

# Proportion contamination estimée dans chaque groupe
prop_N <- mean(grp_N$V1 == 1)
prop_A <- mean(grp_A$V1 == 1)
prop_B <- mean(grp_B$V1 == 1)
```

```r
resultats_groupes <- data.frame(
  Groupe = c("N", "A", "B"),
  Nombre_UE = c(
    nrow(grp_N),
    nrow(grp_A),
    nrow(grp_B)
  ),
  Proportion_contaminee = c(
    prop_N,
    prop_A,
    prop_B
  )
)

print(resultats_groupes)
```

```
##   Groupe Nombre_UE Proportion_contaminee
## 1      N      5287             0.2212975
## 2      A      5244             0.1859268
## 3      B      5301             0.3057914
```

On observe une différence dans la proportion de contamination. On devra alors réaliser un échantillonnage stratifié sans remise. La contamination présupposée du groupe A est de 18.5%, celle du groupe B de 30.5% et du groupe N de 22.1%.

```r
# Lecture des données
srs_pop <- read.table(file = "Detection_Table(1).csv",
                      header = FALSE,
                      sep = ",")

# Taille totale de la population et de l'échantillon
nbgr <- dim(srs_pop)[1]
# On défini les groupes :
Grp_A_tot <-  grp_A[apply(grp_A[, 3:10] == 1, 1, any), ]
Grp_B_tot <-  grp_B[apply(grp_B[, 3:10] == 1, 1, any), ]
Grp_N_tot <-  grp_N[apply(grp_N[, 3:10] == 1, 1, any), ]
```

```r
# Ajouter une variable indiquant le groupe de chaque individu
Grp_A_tot$Groupe <- "A"
Grp_B_tot$Groupe <- "B"
Grp_N_tot$Groupe <- "N"

# Réunir les trois groupes
pop_filtre <- rbind(Grp_A_tot, Grp_B_tot, Grp_N_tot)

# Taille totale de la population filtrée
nbgr <- nrow(pop_filtre)

print(nbgr)
```

```
## [1] 13334
```

```r
# Effectifs de chaque strate
tab_strates <- table(pop_filtre$Groupe)

print(tab_strates)
```

```
##
##    A    B    N
## 4344 4566 4424
```

```r
# Allocation proportionnelle
ech_eff <- round(tab_strates * nbsp / nbgr)

# Corriger les arrondis pour obtenir exactement 150 individus
while (sum(ech_eff) != nbsp) {
  if (sum(ech_eff) < nbsp) {
    i <- which.max(tab_strates * nbsp / nbgr - ech_eff)
    ech_eff[i] <- ech_eff[i] + 1
  } else {
    i <- which.max(ech_eff - tab_strates * nbsp / nbgr)
    ech_eff[i] <- ech_eff[i] - 1
  }
}

print(ech_eff)
```

```
##
##  A  B  N
## 49 51 50
```

```r
library(sampling)

# Échantillonnage stratifié sans remise
ech_res <- strata(
  pop_filtre,
  stratanames = "Groupe",
  size = as.vector(ech_eff),
  method = "srswor"
)

# Extraire les individus sélectionnés
ech_val <- getdata(pop_filtre, ech_res)

# Vérifier le nombre d'individus sélectionnés
nrow(ech_val)
```

```
## [1] 150
```

```r
# Vérifier la répartition par strate
table(ech_val$Groupe)
```

```
##
##  A  B  N
## 49 51 50
```

```r
# Extraction des effectifs réels par strate
N_h <- as.numeric(tab_strates)
names(N_h) <- names(tab_strates)

# Effectifs échantillonnés par strate
n_h <- as.numeric(ech_eff)
names(n_h) <- names(ech_eff)
```

```r
# Proportion de présence présupposée dans chaque strate
p_h <- tapply(ech_val$V1,
              ech_val$Groupe,
              mean)

print(p_h)
```

```
##         A         B         N
## 0.2448980 0.3333333 0.2000000
```

```r
# Estimation globale pondérée de la proportion
poids <- N_h / nbgr

p_global <- sum(poids[names(p_h)] * p_h)

# Estimation du nombre d'UE dont la présence est présupposée
# dans la population filtrée
N_estime <- nbgr * p_global

print(p_global)
```

```
## [1] 0.2602847
```

```r
print(N_estime)
```

```
## [1] 3470.637
```

```r
# Variance estimée de la proportion dans chaque strate
S2_h <- tapply(ech_val$V1,
               ech_val$Groupe,
               var)

# Correction de population finie
f_h <- n_h / N_h

# Variance de l'estimateur stratifié
var_global <- sum(
  (poids[names(p_h)]^2) *
    (1 - f_h[names(p_h)]) *
    S2_h[names(p_h)] /
    n_h[names(p_h)],
  na.rm = TRUE
)

# Erreur-type
se_global <- sqrt(var_global)

# Intervalle de confiance approximatif à 95 %
IC_p <- p_global + c(-1, 1) * 1.96 * se_global

# Limiter les bornes à [0, 1]
IC_p <- pmax(0, pmin(1, IC_p))

# Estimation du nombre total d'UE
N_estime <- nbgr * p_global
```

```r
# Intervalle de confiance du nombre total
IC_N <- nbgr * IC_p

# Affichage des résultats
cat("Proportion estimée :", p_global, "\n")
```

```
## Proportion estimée : 0.2602847
```

```r
cat("Erreur-type :", se_global, "\n")
```

```
## Erreur-type : 0.03570712
```

```r
cat("IC 95 % de la proportion :",
    IC_p[1], "-", IC_p[2], "\n")
```

```
## IC 95 % de la proportion : 0.1902988 - 0.3302707
```

```r
cat("Nombre estimé d'UE :", N_estime, "\n")
```

```
## Nombre estimé d'UE : 3470.637
```

```r
cat("IC 95 % du nombre d'UE :",
    IC_N[1], "-", IC_N[2], "\n")
```

```
## IC 95 % du nombre d'UE : 2537.444 - 4403.829
```

```r
# UE échantillonnées dont la présence est présupposée absente
UE_absentes <- ech_val[ech_val$V1 == 0, ]

# Résultats des huit essais de détection
det_absentes <- as.matrix(UE_absentes[, 3:10])

# Nombre total de tests
nb_tests <- length(det_absentes)

# Nombre de résultats positifs
nb_positifs <- sum(det_absentes == 1)

# Estimation de la probabilité de faux positifs
p_fp_estime <- nb_positifs / nb_tests

# Intervalle de confiance binomial exact à 95 %
IC_fp <- binom.test(
  nb_positifs,
  nb_tests,
  conf.level = 0.95
)$conf.int

# Affichage
cat("Probabilité estimée de faux positifs :",
    p_fp_estime, "\n")
```

```
## Probabilité estimée de faux positifs : 0.2308559
```

```r
cat("Intervalle de confiance à 95 % :",
    IC_fp[1], "-", IC_fp[2], "\n")
```

```
## Intervalle de confiance à 95 % : 0.2035082 - 0.2600018
```

```r
cat("Probabilité théorique : 0.18\n")
```

```
## Probabilité théorique : 0.18
```

On observe une probabilité de faux positif de 21.7% à un intervalle de confiance de 95%, cette valeur dépasse la probabilité théorique de 18%. Cela pourrait venir du calcul utilisé.